DEPARTMENT OF
NETWORKED SYSTEMS
AND SERVICES

Implementációs hibák

VIHIAC07 – IT Security, 2026

Dr. András Gazdag

CrySyS Lab, BME

agazdag@crysys.hu

www.crysys.hu

MŰEGYETEM 1782

Áttekintés

- Bevezetés a memóriakezelési sérülékenységekbe
- x86 architektúra
- Függvényhívási mechanizmusok
- Stack overflow
- Védelmi mechanizmusok

Szoftverbiztonság

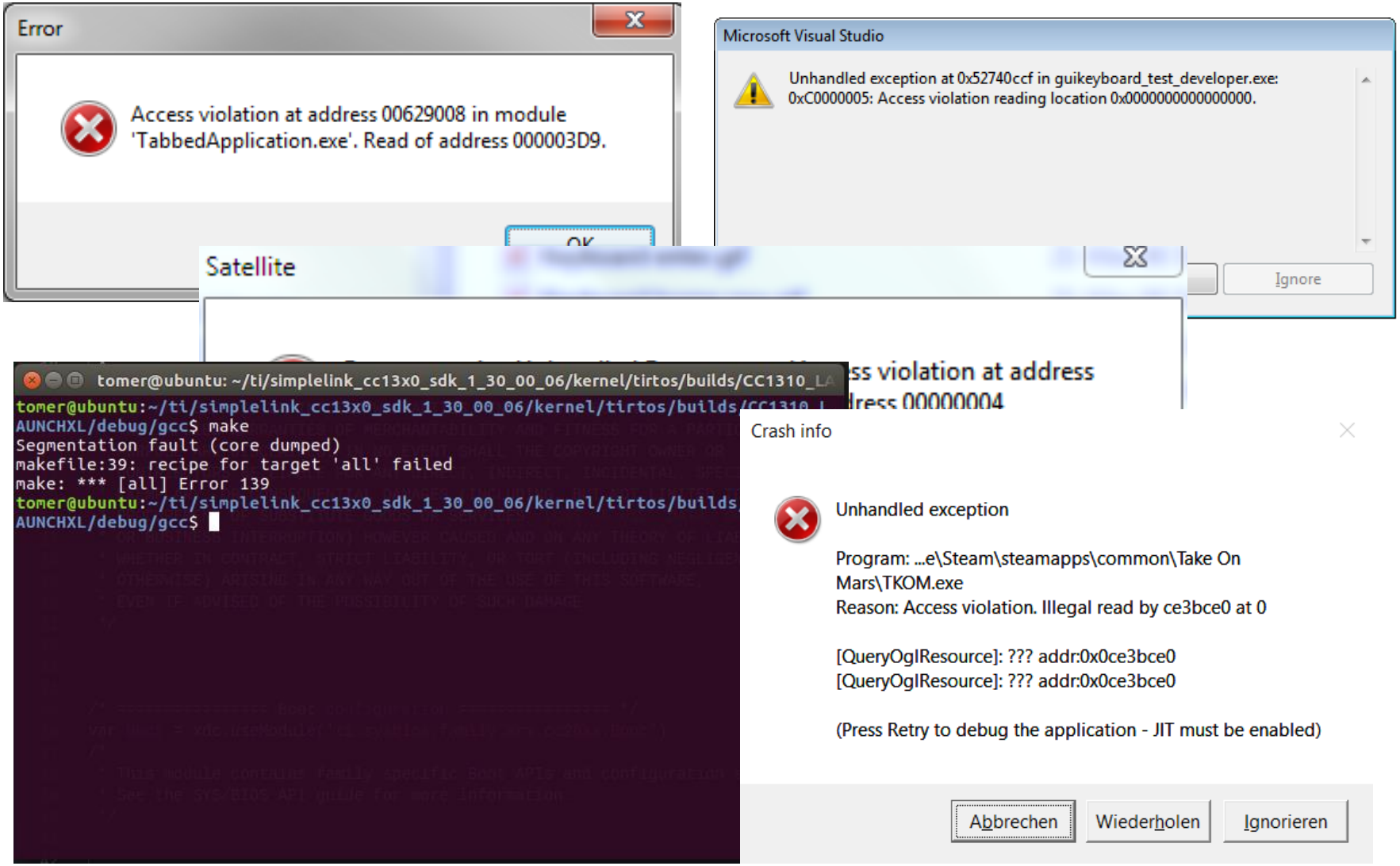
- Mit jelent az, hogy egy szoftver biztonságos?
  - Az egyik lehetséges definíció szerint egy szoftverrendszer akkor biztonságos, ha teljesíti a meghatározott vagy feltételezett biztonsági célokat.
- A különböző biztonsági követelmények ellentétbe kerülhetnek egymással
  - Pl. egy támadás észlelésekor a rendszer zárolása jó a titkosság és az integritás szempontjából, de rossz a rendelkezésre állás szempontjából.

Sérülékenységi kategóriák

- Memóriakezelési sérülékenységek
  - térbeli sérülékenységek
  - időbeli sérülékenységek
- Strukturált kimenet-generálási sérülékenységek
  - Pl. SQL injection, command injection, XSS, stb.
- Versenyhelyzet (Race Condition) sérülékenységek
  - Ellenőrzési idő – felhasználási idő (TOCTOU)
- API sérülékenységek
  - az implementációs sérülékenységek általánosítása szerződésszegésként értelmezve
- Side-channel sérülékenységek
  - Tipikus probléma a kriptográfia területén

Memóriakezelési sérülékenységek

Memória-korrupció



Memória-korrupció

- Akkor következik be, ha egy memóriacím tartalma programozási hibák miatt nem szándékos módon módosul
  - a módosítás lehet véletlen vagy szándékos (egy támadás részeként)
- Ha a módosult memóriatartalmat használja később a program az *összeomláshoz* vagy *definiálatlan viselkedéshez* vezethet
- Memória-korrupció példák:
  - Buffer overflow (stack-en vagy heap-en)
  - Integer overflow
  - NULL pointer vagy definiálatlan pointer (felszabadított memóriára mutatnak)

Puffertúlcsordulás (Buffer overflow)

- Akkor fordul elő, ha egy memória foglalás határát átlépi az adat írási művelet során
  - Tipikus eset: több adatot másolunk egy bufferbe, mint amennyi helyet foglaltunk
- Kihasználható arra, hogy a helyes működés szempontjából kritikus területeket módosítson egy támadó (visszatérési cím, pointerek, változó értékek)
- Felhasználható a program vezérlési folyamatának megváltoztatására
  - A program elkezdheti végrehajtani a beszúrt rosszindulatú kódot

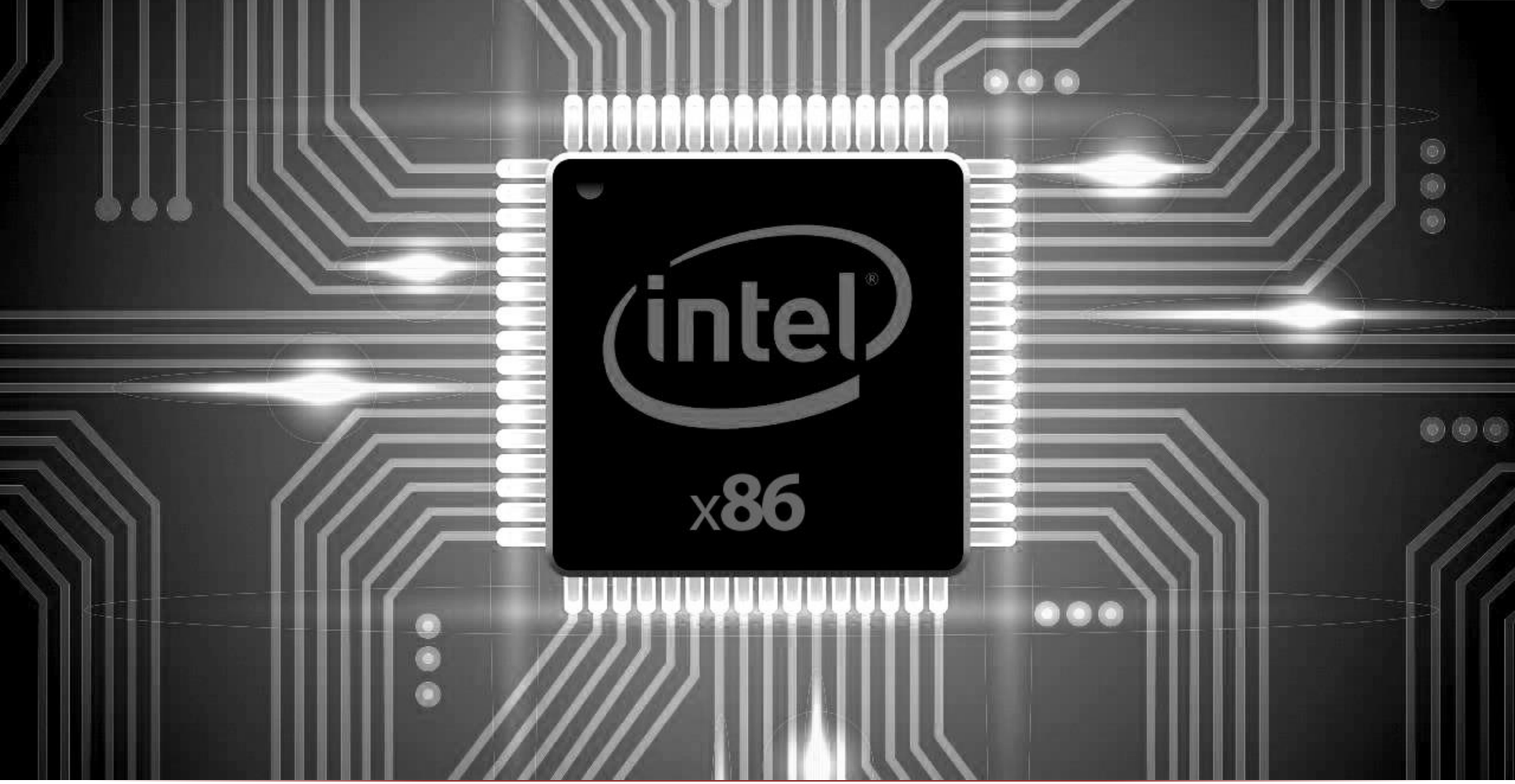
Stack overflow

- A puffertúlcsordulás speciális formája

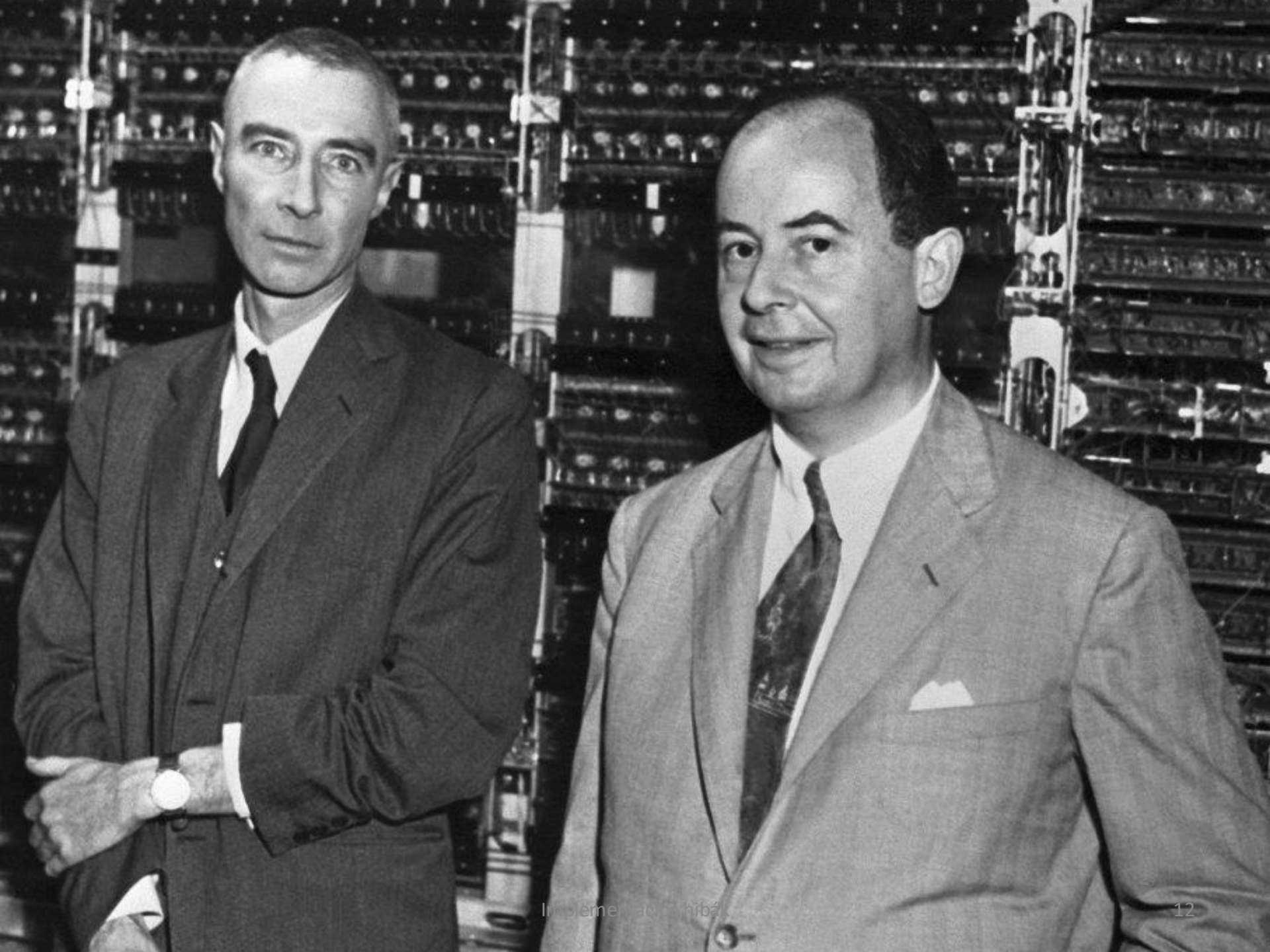
- Támadás menete:
  - Akkor fordul elő, ha egy függvényhívás során egy felhasználótól érkező adatot egy stacken található lokális változóba másolunk
  - A bemásolt adat így felülírja a stacken tárolt kritikus értékeket, első sorban a *visszatérési címet*
  - Amikor a függvény visszatér, akkor a módosított értéket használja a CPU, és így a végrehajtás nem ott folytatódik, ahol normálisan kellene

Túlcsordulási támadások története

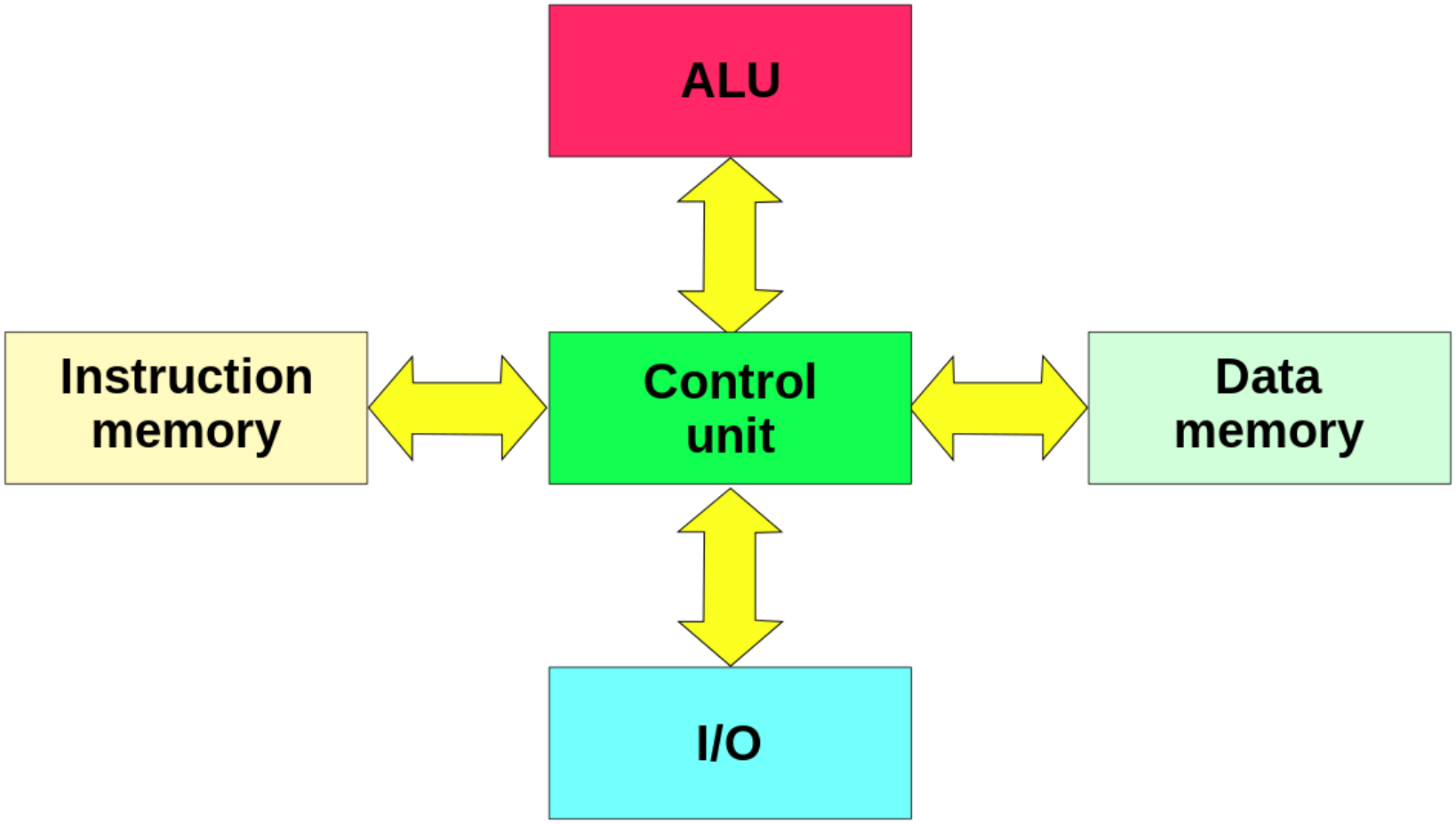
- Morris worm (1988): overflow in fingerd
  - 6000 gép fertőződött meg (az internet 10%-a)
- CodeRed (2001): overflow in Microsoft IIS
  - 300 000 gép fertőződött meg 14 óra alatt
- SQL Slammer (2003): túlcsordulás az MS-SQL szerverben
  - 75 000 gép (az összes sérülékeny internethez kapcsolt gép 90%-a) fertőződött meg 10 perc alatt
- 2006-2008 között az újonnan felfedezett sérülékenységek több mint 50%-a puffertúlcsordulással kapcsolatos volt
- Ma a webes sérülékenységek kihasználása elterjedtebb, azonban a memória korrupciós sérülékenységeket még mindig nagy mértékben kihasználják



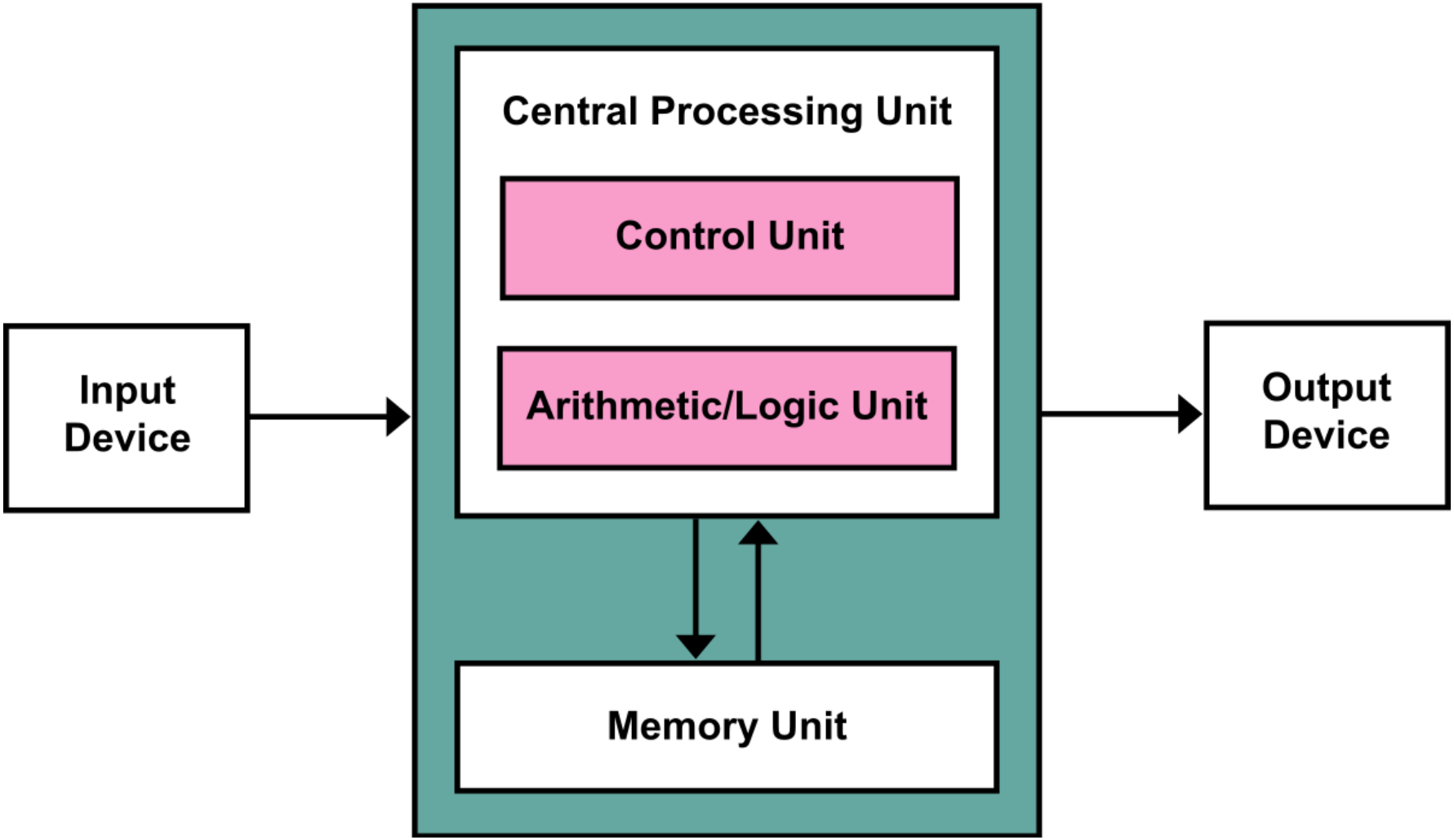
x86 architektúra



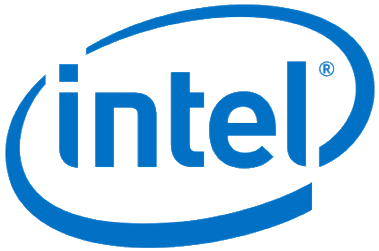
Harvard architektúra



Neumann architektúra



Intel x86 – főbb regiszterek



- Regiszter: 32 bites tároló a mikroprocesszoron belül
- Általános regiszterek: **`eax`**, **`ebx`**, **`ecx`**, **`edx`**, **`esi`**, **`edi`**, **`stb`**.
- Speciális célú regiszterek
  - **`esp`** – stack pointer
  - **`ebp`** – stack frame (base) pointer
  - **`eip`** – instruction pointer (a következőnek végrehajtandó utasítás címét tartalmazza)

Intel x86 – utasítások

Name	Description
MOV <dest>, <src>	Moves data from <src> to <dest> <dest> := <src>
ADD <op1>, <op2>	Adds two integers <op1> := <op1> + <op2>
SUB <op1>, <op2>	Subtracts an integer from another <op1> := <op1> - <op2>
CMP <op1>, <op2>	Compares two integers and sets the flags, just like SUB <op1>, <op2>
AND <op1>, <op2>	Bitwise AND of two integers <op1> := <op1> AND <op2>
OR <op1>, <op2>	Bitwise OR of two integers <op1> := <op1> OR <op2>
XOR <op1>, <op2>	Exclusive OR of two integers <op1> := <op1> XOR <op2>

Intel x86 – flagek

- Stored in the status register, called FLAGS (F)

Name	Description
ZF – zero flag	Set to 1 if the result of the last arithmetic operation was zero
CF – carry flag	Set to 1 if the last arithmetic operation caused carry or borrow in the most significant bit
SF – sign flag	Set to 1 if the result of the last arithmetic operation was a negative value
OF – overflow flag	Set to 1 if the last arithmetic operation caused overflow
PF – parity flag	Set to 1 if the least significant byte of the result of the last arithmetic operation contains an even number of 1 bits

Intel x86 – vezérlő utasítások

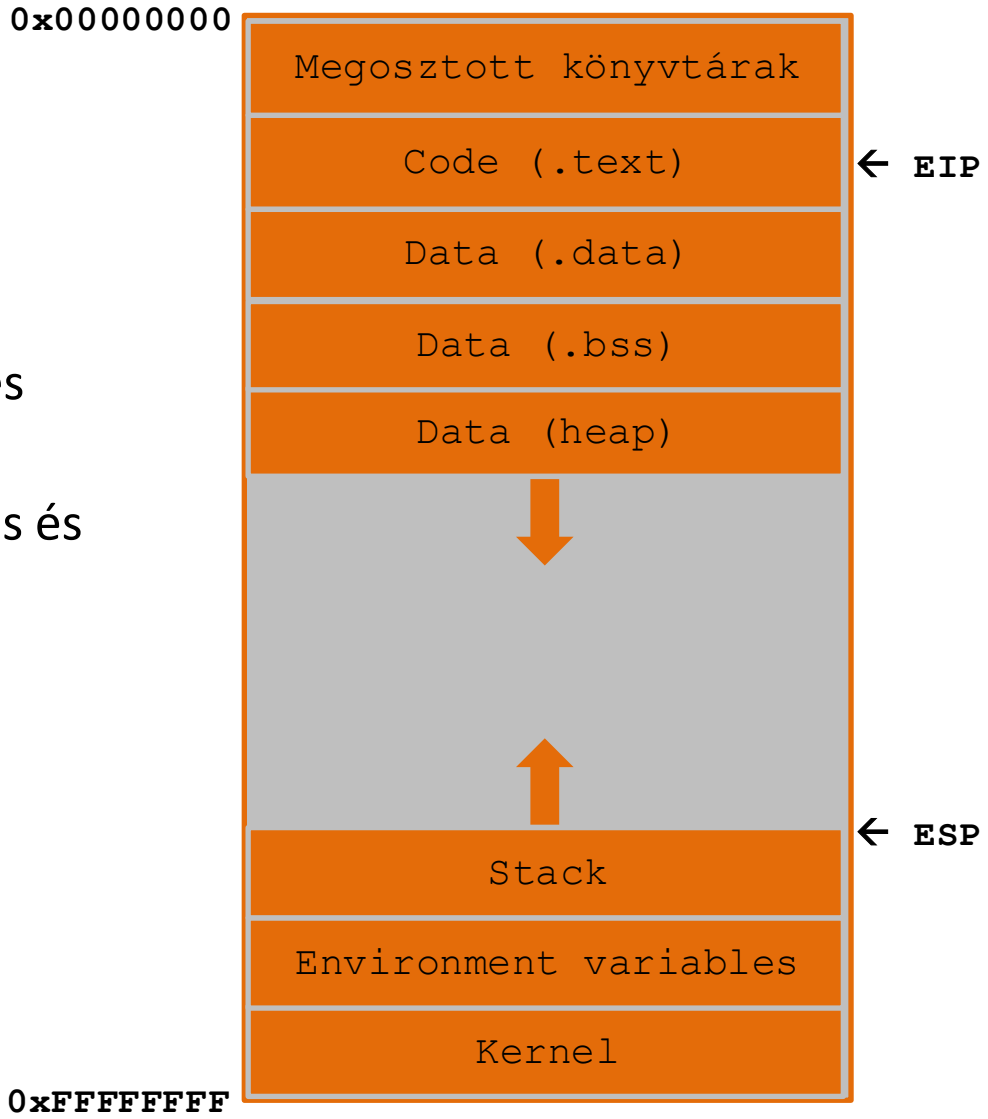
Name	Description
JMP <addr> or JMP <offs>	Jump, i.e. continues the code execution from a new address (absolute or relative) eip:=<addr> or eip:=eip + <offs>
J<cond> <offs>	Jumps if condition is true (otherwise continues with next instr.) if (cond) then eip:=eip + <offs> Some possible conditions: • Z / NZ – zero / non-zero, example: JZ <offs> / JNZ <offs> • E / NE – equals (ZF=1) / not equals (ZF=0) • S / NS – sign, i.e. negative / no sign, i.e. zero or positive • L, NGE (signed) – less, not greater than or equal (SF<>OF) • GE, NL (signed) – greater than or equal, not less (SF=OF) • …

Intel x86 – stack és hívási műveletek

Name	Description
PUSH <value>	Places <value> on top of stack [esp] esp := esp - 4 [esp] := <value>
POP <dest>	Loads a value from the top of stack to <dest> <dest> := [esp] esp := esp + 4
LEAVE	Used at the end of functions, shortcut for MOV esp, ebp POP ebp
CALL <addr> CALL <offs>	Stores the return address on the stack and calls (relative or absolute jump) a function PUSH eip JMP <add> / JMP <offs>
RET	Loads the top of the stack into the eip instruction pointer (and so jumps back / to the return address) POP eip

Memória elrendezés – szegmensek

- `Code` szegmens:
  - Végrehajtható utasítások
  - Általában csak olvasható
- `Data` szegmens:
  - `.data`: inicializált globális és statikus változók
  - `.bss`: inicializálatlan globális és statikus változók
  - `heap`: dinamikusan kezelt memóriaterület
- `Stack` szegmens:
  - Lokális változók
  - Függvényhíváshoz szükséges értékek

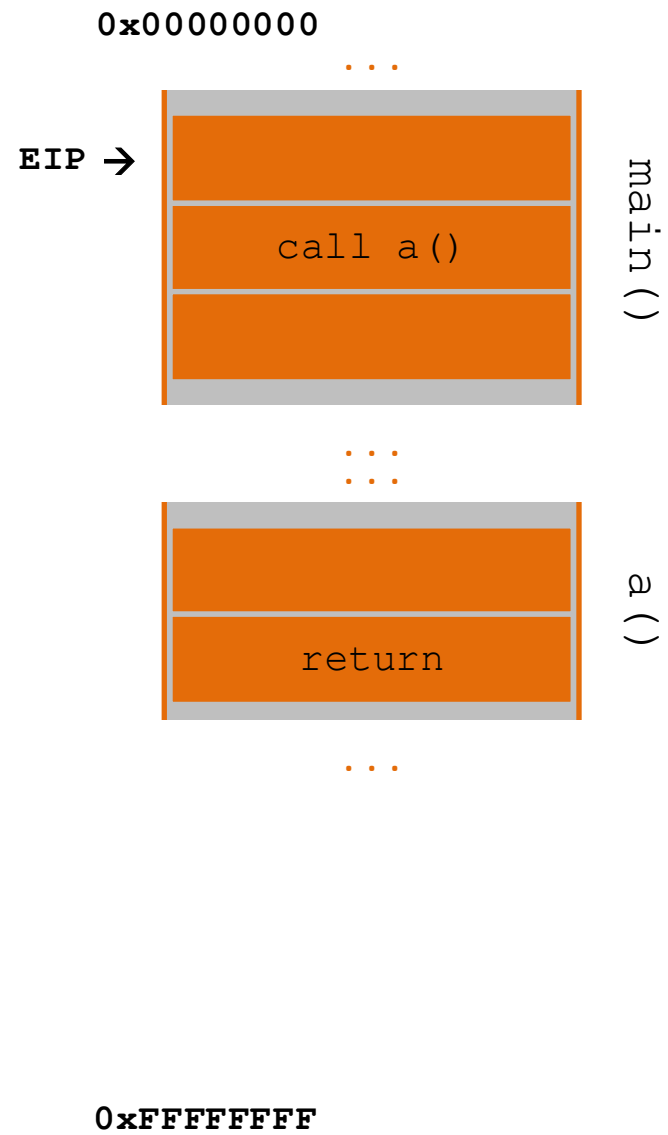


Függvényhívási mechanizmus

Függvényhívás – Hová térjünk vissza?

EIP →

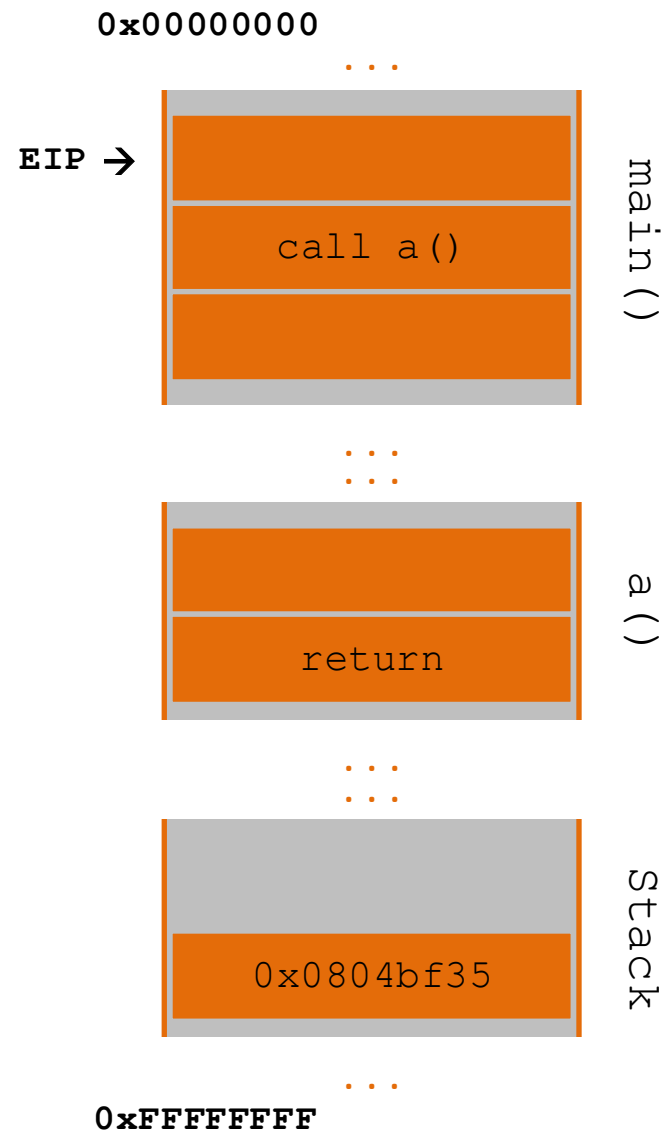
```
1 int main() {
2   // ...
3   a();
4   // ...
5 }
6
7 void a() {
8   //...
9 }
```



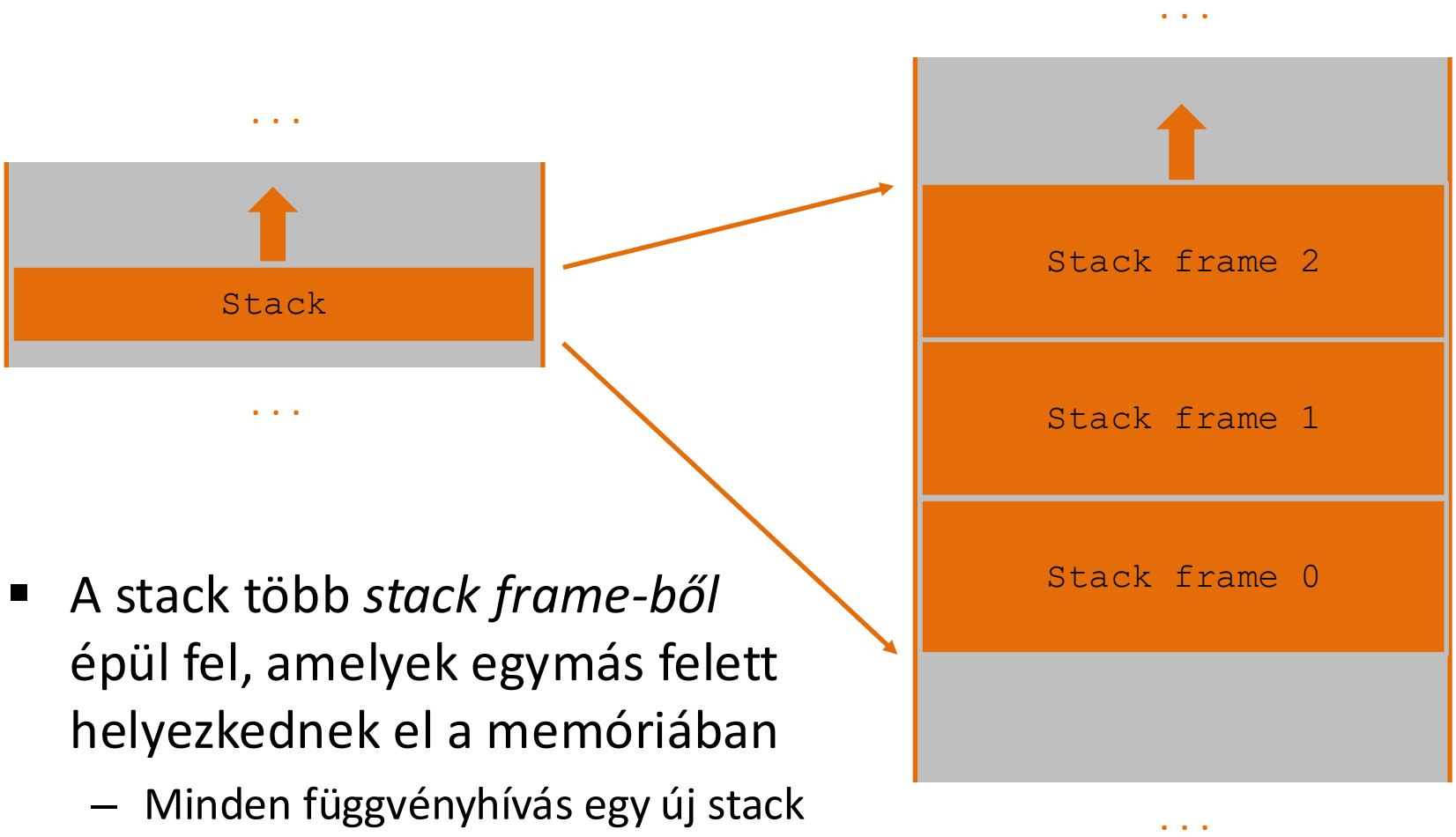
Függvényhívás – Visszatérési cím

EIP →

```
1 int main() {
2   // ...
3   a();
4   // ...
5 }
6
7 void a() {
8   //...
9 }
```



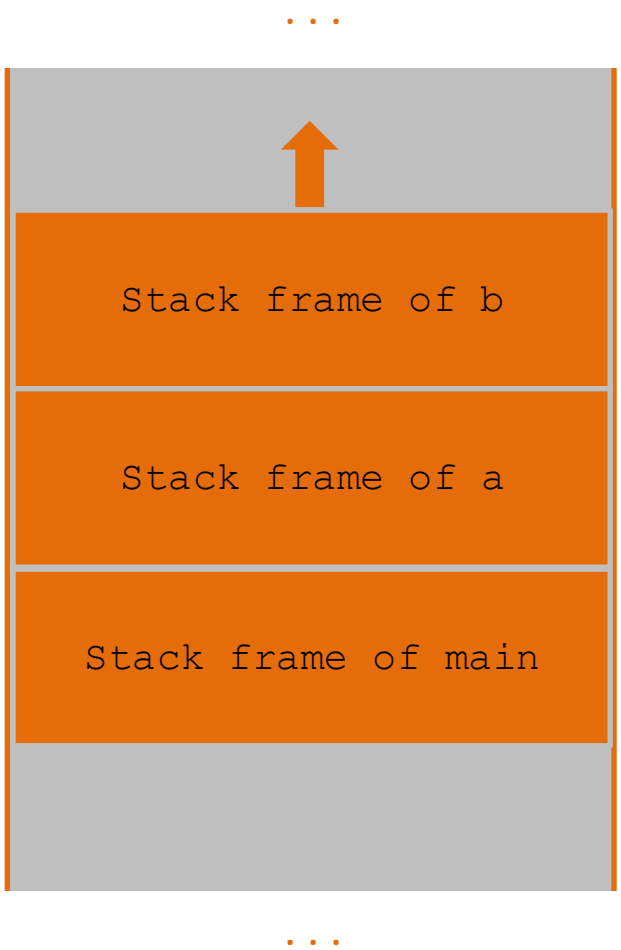
Stack felépítése



- A stack több *stack frame-ből* épül fel, amelyek egymás felett helyezkednek el a memóriában
  - Minden függvényhívás egy új stack frame-et hoz létre

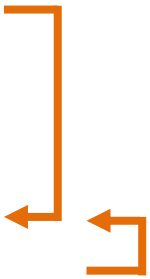
Egymásba ágyazott hívások stack frame-je

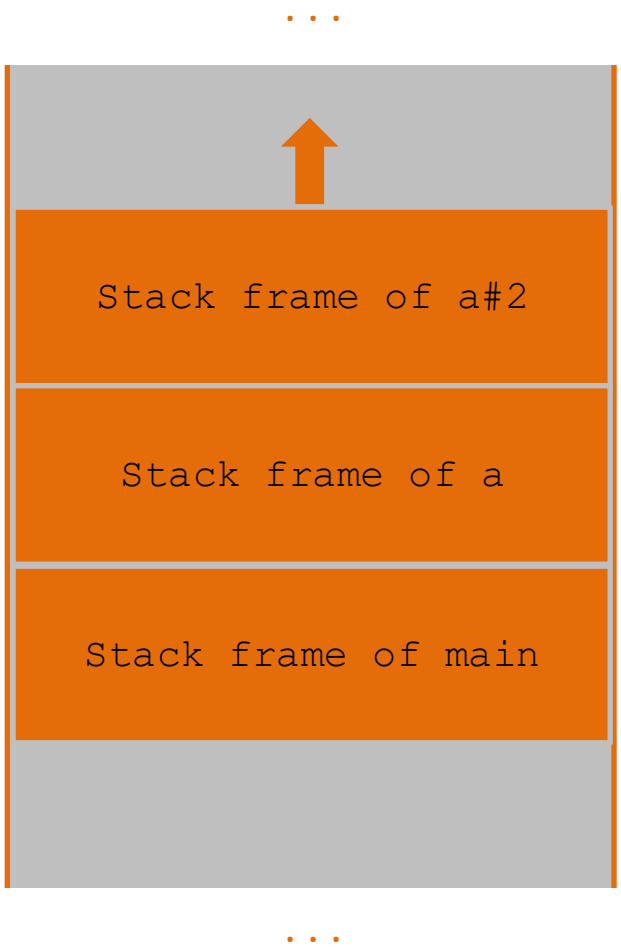
```
1  int main() {
2    a();
3    // ...
4  }
5
6  void a() {
7    b();
8    //...
9  }
10
11 void b() {
12   //...
13 }
```



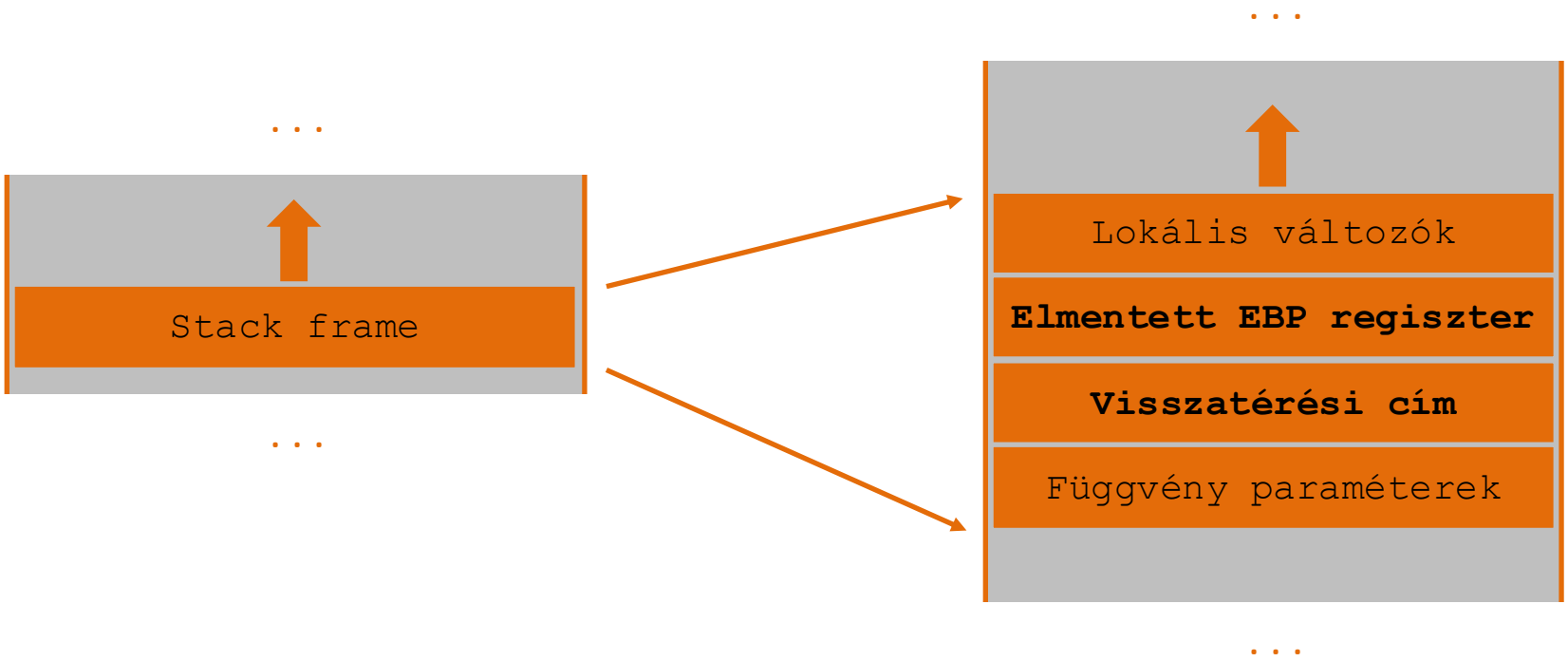
Rekurzív függvényhívások stack frame-je

```
1 int main() {
2   a();
3   // ...
4 }
5
6 void a() {
7   a();
8   //...
9 }
```





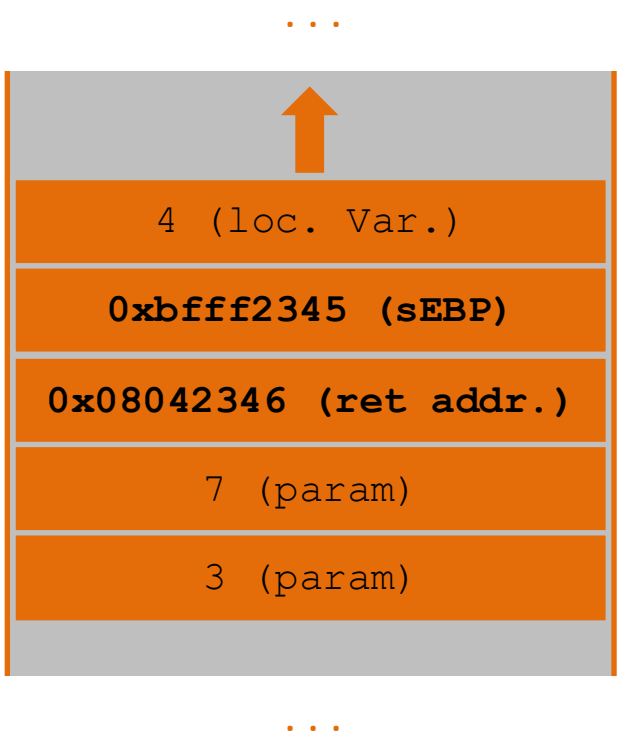
A stack frame szerkezete



Függvényhívás – a stack működése

EIP →

```
1  int main(void) {
2      int a, b, c;
3      a=7;
4      b=3;
5      c = addnum(a,b);
6      printf("Result is: %d", a+b);
7  }
8
9  int addnum(int a, int b) {
10     int c = 4;
11     c = a + b;
12     return c;
13 }
```



Hívási konvenciók

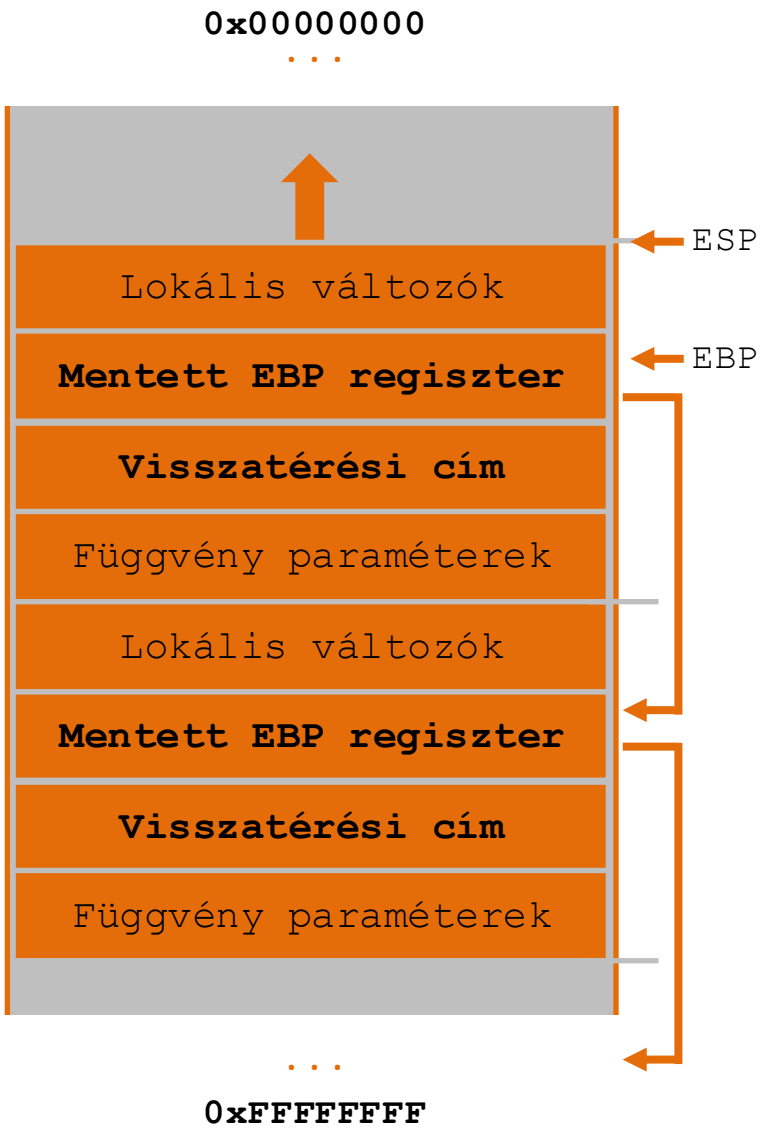
- x86 hívási konvenciók
  - **`cdecl`**: az előbb leírt folyamat; a hívó feladata a paraméterek eltávolítása, a paraméterek jobbról balra sorrendben kerülnek elhelyezésre
  - **`fastcall`**: hasonló a cdeclhöz, de néhány pareméter (tipikusan az első kettő), regiszterekben vannak átadva (MS VC and GCC)
  - **`thiscall`**: C++-ban használatos, a this pointer is automatikusan átadásra kerül

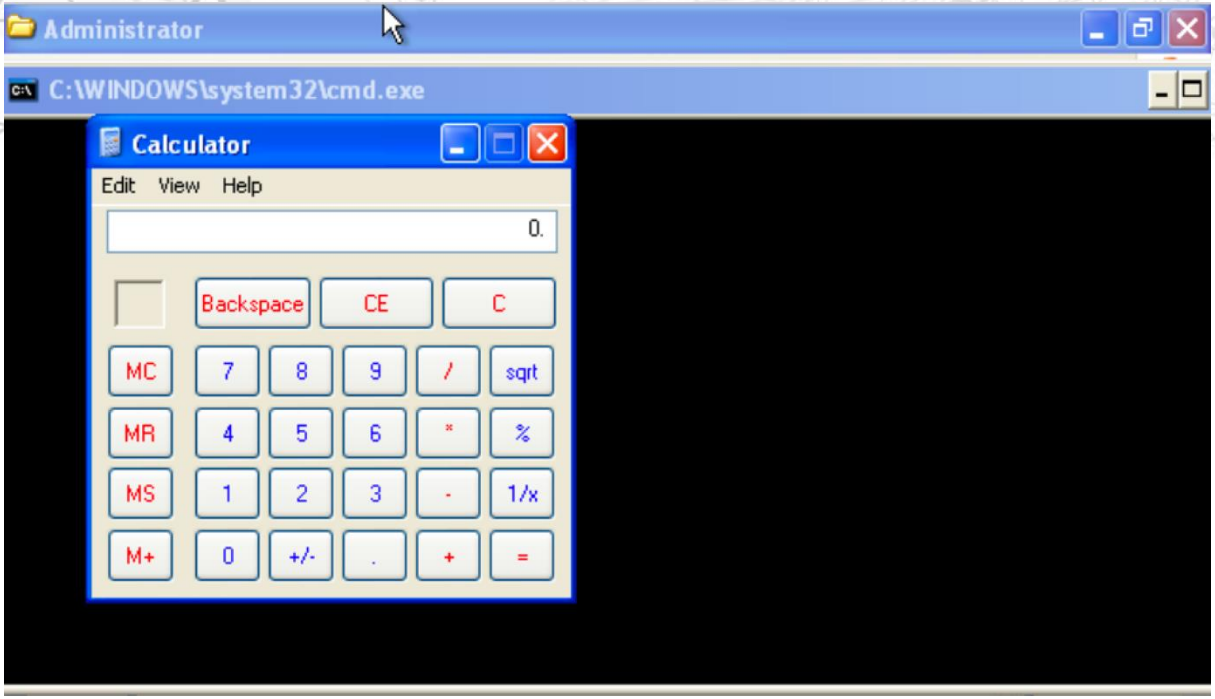
- x64 hívási konvenciók
  - **`Microsoft x64:`** az első 4 paraméter regiszterekben kerül átadásra, a további paraméterek itt is a stacken vannak elhelyezve, külön regiszterek tárolják a lebegőpontos számokat
  - **`System V AMD64 ABI:`** ezt használja a Solaris, Linux, FreeBSD és macOS; az első 6 paraméter regiszterekben kerül átadásra, a további paraméterek itt is a stacken vannak elhelyezve, külön regiszterek tárolják a lebegőpontos számokat

- És még sok más konvenció létezik! (pl. ARM, PowerPC, SPARC...)

Az EBP regiszter szerepe

- Az EBP regiszter mindig az aktuálisan használt stack frame közepére mutat
  - Nem a stack végére! (Az az ESP)
  - Minden függvényhíváskor frissíteni kell
- Oda mutat, ahol a hívó függvény elmentett EBP értéke található a stack frame-en belül
  - Lokális változók helye: EBP-x
  - Visszatérési cím helye: EBP+4
  - Paraméterek helye: EBP+8, EBP+12, …





A stack overflow támadás

Stack overflow

- A stack overflow akkor következik be, ha egy függvény a felhasználó által kontrollált adatot másol egy stack-en található lokális pufferbe a méret ellenőrzése nélkül
- Veszélyes függvények
  - strcpy, sprintf, strcat, gets, fgets, memcpy, …
- A másolt adat felülírja a stacken található értékeket egészen a visszatérési címig
- Amikor a függvény visszatér, az EIP regiszter értéke a támadó által választott érték lesz → a támadó kontrollálja innentől a végrehajtás menetét
- Ha a támadó tud kódot elhelyezni a memóriában, akkor az EIP megfelelő módosítása után a támadó kódja kerül végrehajtásra

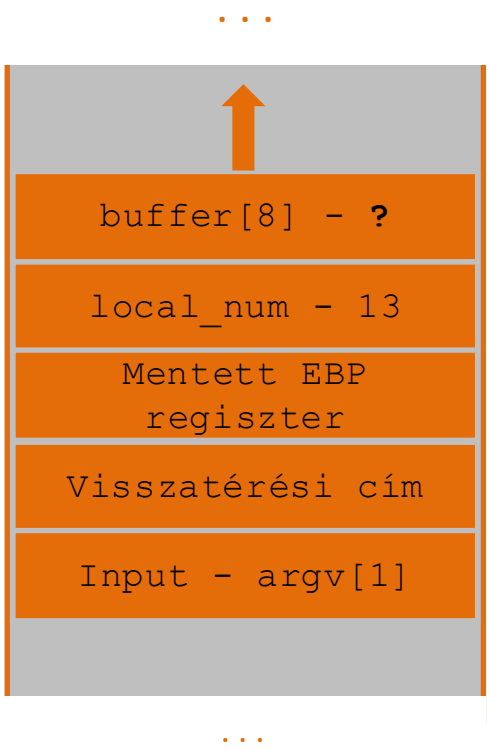
Stack overflow sérülékenység

```c
#include <stdio.h>
#include <string.h>

void vulnerable_function(char *input) {
  int local_num = 13;
  char buffer[8];
  strcpy(buffer,input);
  printf("Local num is: %d\n", local_num);
  printf("Buffer is: %s\n", buffer);
}

int main(int argc, char* argv[]) {
  int k=3;
  vulnerable_function(argv[1]);
  return 0;
}
```

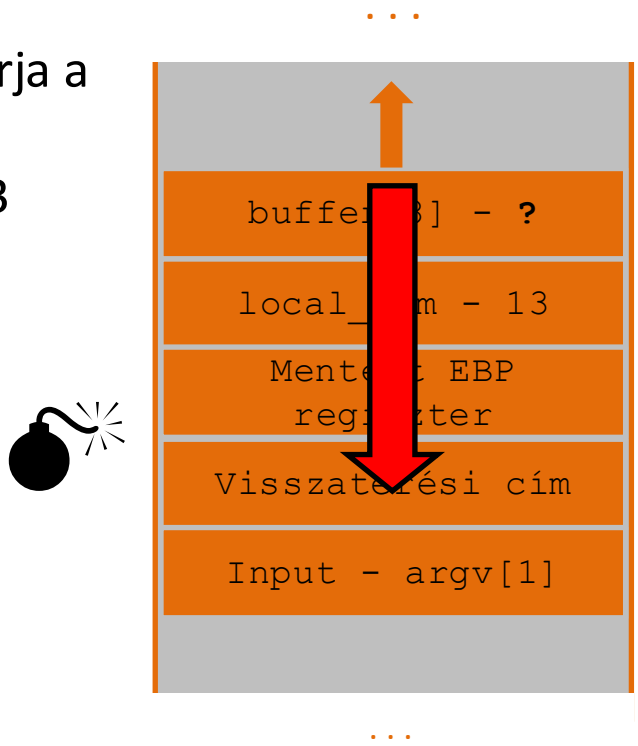
A puffer túlcsordulhat!



A visszatérési cím felülírása

- Hiányzik a megfelelő méret ellenőrzés!
  - Egy hosszú bemenet hatására az strcpy túlírja a lokális puffert
    Példa bemenet: ABABABABABABABABABAB

- A visszatérési cím így módosul
  - Ez kihasználható

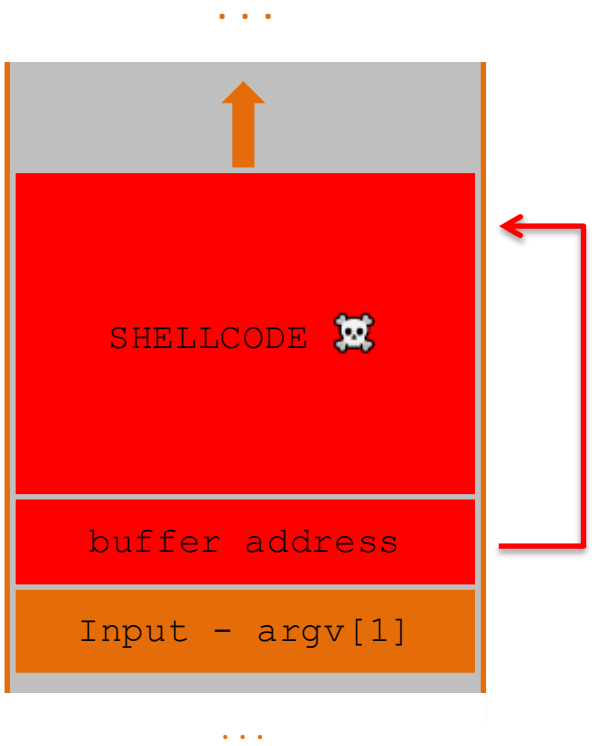



```
[92801.217602] vulnerable_app[29676]:
    segfault at 41424142
    ip 41424142
    sp bffff320
    error 4
```

Stack overflow – egyedi shellkód

```
1  #include <stdio.h>
2  #include <string.h>
3
4  void vulnerable_function(char *input) {
5    char buffer[100];
6    strcpy(buffer,input);
7    printf("Buffer is: %s\n", buffer);
8  }
9
10 int main(int argc, char* argv[]) {
11   vulnerable_function(argv[1]);
12   return 0;
13 }
```

EIP →



Stack overflow

- Hová helyezzük a shellkódot?
  - Lokális változóként tárolt pufferbe (esetleg NOP sleddel kiegészítve)
    - » Sikerességhez kell, hogy ismert legyen a változó címe,
    - » A stacknek futtathatónak kell lennie
  - Környezeti változókba
    - » Könnyen megvalósítható
    - » Kisebb méretű kód esetén praktikus
    - » A stacknek futtathatónak kell lennie
  - Bármilyen *írható* és *futtatható* területre

- `NOP sled`: a támadó kód elé lehet célszerű elhelyezni az ugrás „megkönnyítéséhez”
  - Alkalmazásának oka: nagyobb eséllyel találjuk el a shellkódunkat
  - A legegyszerűbb megoldás: 0x90 (no operations - NOP) utasítás

Védekezés

Védelmi stratégiák

- A szoftver formális helyességbizonyítása
  - garantálja, hogy a kód hibamentes (egy adott specifikáció szerint)
- A szoftver újraírása biztonságos programozási nyelven
- Szoftvertesztelés
  - A szoftverhibák felfedezése, mielőtt kárt okozhatnának
- Kerülőmegoldások (Mitigációk)
  - Részlegesen védik a rendszert javítatlan vagy ismeretlen sérülékenységek esetén is

Védelmi stratégiák – Formális verifikáció

- A szoftververifikáció egy adott specifikáció szerint igazolja a kód helyességét
- A formális verifikáció különböző formái léteznek:
  - korlátozott modellellenőrzés
  - absztrakt értelmezés
- Mindegyik igazolja, hogy egy adott kódrészlet megfelel a formális specifikációnak, és garantálja, hogy a biztonsági szabályok megsértése nem lehetséges

Védelmi stratégiák – Nyelvi megoldás

- Cél: biztonsági tulajdonságok kikényszerítése a programozási nyelv részeként, megvédve a programozót a hibáktól

- *Javaban* kényszerként jelenik meg a típusok alkalmazása (erősen típusos nyelv) és emellett a JVM garantálja a memória biztonságot

- *Rust* is kikényszeríti a memória biztonságot, valamint garantálja, hogy nem fordulhat elő versenyhelyzet
  - A Rust egyértelmű tulajdonosi modellje megtiltja az egyidejű módosítást, és lehetővé teszi a fordítónak, hogy fordítási időben ellenőrizze a memória tulajdonosát
  - A Rust erős memóriabiztonsági, típusbiztonsági és versenyhelyzet-mentességi garanciákat nyújt elhanyagolható teljesítményköltséggel, a meredek tanulási görbe árán – a programozónak minden tulajdonosi feltételezést explicit módon kell megjelenítenie a kódban

Védelmi stratégiák – Tesztelés

- Ez a feladat kiegészíti a szoftverfejlesztés folyamatát, és ha helyesen végzik, beépül a fejlesztési folyamatba a folyamatos szoftvertesztelés érdekében
  - A funkcionális és működési közbeni követelmények jól tesztelhetőek

- A biztonsági követelmények általában közvetlenül nem tesztelhetőek: egy sérülékenység hiányát nehéz igazolni
  - C/C++ alkalmazások esetén közvetve tesztelhetjük, hogy a memóriabiztonsági és típusbiztonsági garanciák teljesülnek-e, a program tesztelés közbeni viselkedését megfigyelve
    - » Ahelyett, hogy egy eredmény helyes kiszámítását figyelnénk, azt ellenőrizzük, hogy a program összeomlik-e vagy biztonsági kivétel miatt leáll-e a végrehajtás

Védelmi stratégiák – Tesztelés (előzetes...)

- Manuális tesztelés / automatizált tesztelés
  - Tesztvezérelt fejlesztés
  - Folyamatos integrációs rendszer
- Speciális célmegoldások: sanitizerek
  - AddressSanitizer
  - LeakSanitizer
  - MemorySanitizer
- Randomizált tesztelés
  - Fuzzing (AFL)

Védelmi stratégiák – Kerülőmegoldások

- Úgy kell ezekre a lehetőségekre tekinteni, mint egy *utolsó védelmi vonal* a támadóval szemben
  - Célja a memóriakezeléssel kapcsolatos hibák felismerése és/vagy megállítása az alacsony szintű memória biztonsági szabályok betartatásával
  - A logikai hibák kívül esnek ezen módszerek működésén, mivel azok az alkalmazás követelményeitől és specifikációjától függenek
  - Általában némi teljesítménycsökkenést okoznak a kiegészítő ellenőrzések miatt

- Az elavult (legacy) alkalmazások utolsó reménye!
  - Kompatibilitási problémák azért itt is felléphetnek

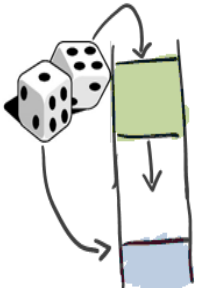
Védelmi stratégiák – Kerülőmegoldások

- A működés alapötlete, hogy teljesítményről mondunk le erősebb biztonságért cserébe
  - Ezek a módszerek javítatlan és/vagy ismeretlen hibák ellen védenek
  - Ezen módszerek futtatásának költsége jól látható a FreeBSD és HardenedBSD teljesítménye között

- Mely rendszerszintű védelmi megoldások merülhetnek fel?

DEP

kanárik



ASLR

DEP – Adatvégrehajtás-megelőzés

- A védelem alapötlete, hogy szeparáljuk el a memóriában az írható területeket a futtathatóaktól
  - Pl. a stack írható legyen, de ne lehessen futtatni
  - A programmemória általában futtatható, de nem kell írhatóknak lennie
    - » kivéve modulok dinamikus betöltésekor!
- Általában a memória management szintjén kerül implementálásra
  - Az adatokat tároló teljes memórialapok nem futtathatóra állíthatók (NX bit)
- Megoldás lehet egy olyan számítógép architektúra is, ami koncepcionálisan is elkülöníti már a különböző típusú adatokat
  - Harvard architektúra!
    - » fizikailag elkülönített memória az adatok és a kód számára (különböző buszok)
  - A CPU egyszerre tud utasításokat és adatokat betölteni
  - Az adatmemória sosem végrehajtható

Kanárik

- Kanárikat bányákban használták régen a veszélyes gázok miatt

- A stack kanári egy olyan érték, amit a elmentett EBP és a lokális változók közé helyezünk el
  - ezt a fordítóban engedélyezni lehet
  - a meglévő programok újrafordítását igényli
- A függvény a visszatérés előtt ellenőrzi ennek a mezőnek a változását
  - Ha módosult, akkor támadás történt, és le kell állítani a végrehajtást!

ASLR - Address Space Layout Randomization

- Címtér egyes részeinek a véletlenszerű elmozgatása
  - Az operációs rendszer véletlenszerűen választja meg a kulcsfontosságú adat- és programterületek memóriabeli pozícióját (stack, .data, .text, megosztott könyvtárak, ...)
- Miért előnyös ez?
  - Puffertúlcsordulás: a támadó nem tudja megjósolni a shellkód címét
  - return-to-libc: a támadó nem tudja megjósolni a célfüggvény címét
  - ROP: a támadó nem tudja megjósolni a gadgetek (kódtöredékek) címét
- Elérhető Linux, macOS és Windows rendszereken is
  - Linux kernel > 2.6.11
  - Windows Vista óta
- Mi a helyzet a beágyazott rendszerekkel és az IoT eszközökkel???

Az ASLR története

- ”the Linux PaX project first coined the term ASLR, and published the first design and implementation of ASLR in July 2001 as a patch for the Linux kernel”

- A PaX csoportot egy *pipacs* nevű magyar „hacker” alapította
  - *Aki valójában a BME-n végzett!*

- pipacs/PaX csoport elnyerte a „Pwnie lifetime achievement” díjat ezért a munkáért 2011-ben
  - A Pwnie díjat a biztonság növelése érdekében végzett kiemelkedő munkával lehet megnyerni
  - A díjakat évente a Black Hat biztonsági konferencián adják át

Az ASLR gyenge pontjai

- 32 bites architektúrákon az ASLR korlátozott a randomizáláshoz rendelkezésre álló bitek száma miatt
  - A 32 bites címnek csak a középső 16 bitje módosítható
  - A 16 bitnyi randomizálás percek alatt megkerülhető brute force támadással
- 64 bites architektúrákon viszont az ASLR egy erős védelem!
  - A 64 bitből 40 bit áll rendelkezésre a randomizáláshoz
  - A 40 bites randomizálás ellen brute force támadás elméletileg lehetséges, de valószínűleg nem marad észrevétlen
    - » A program sokszor fog összeomlani

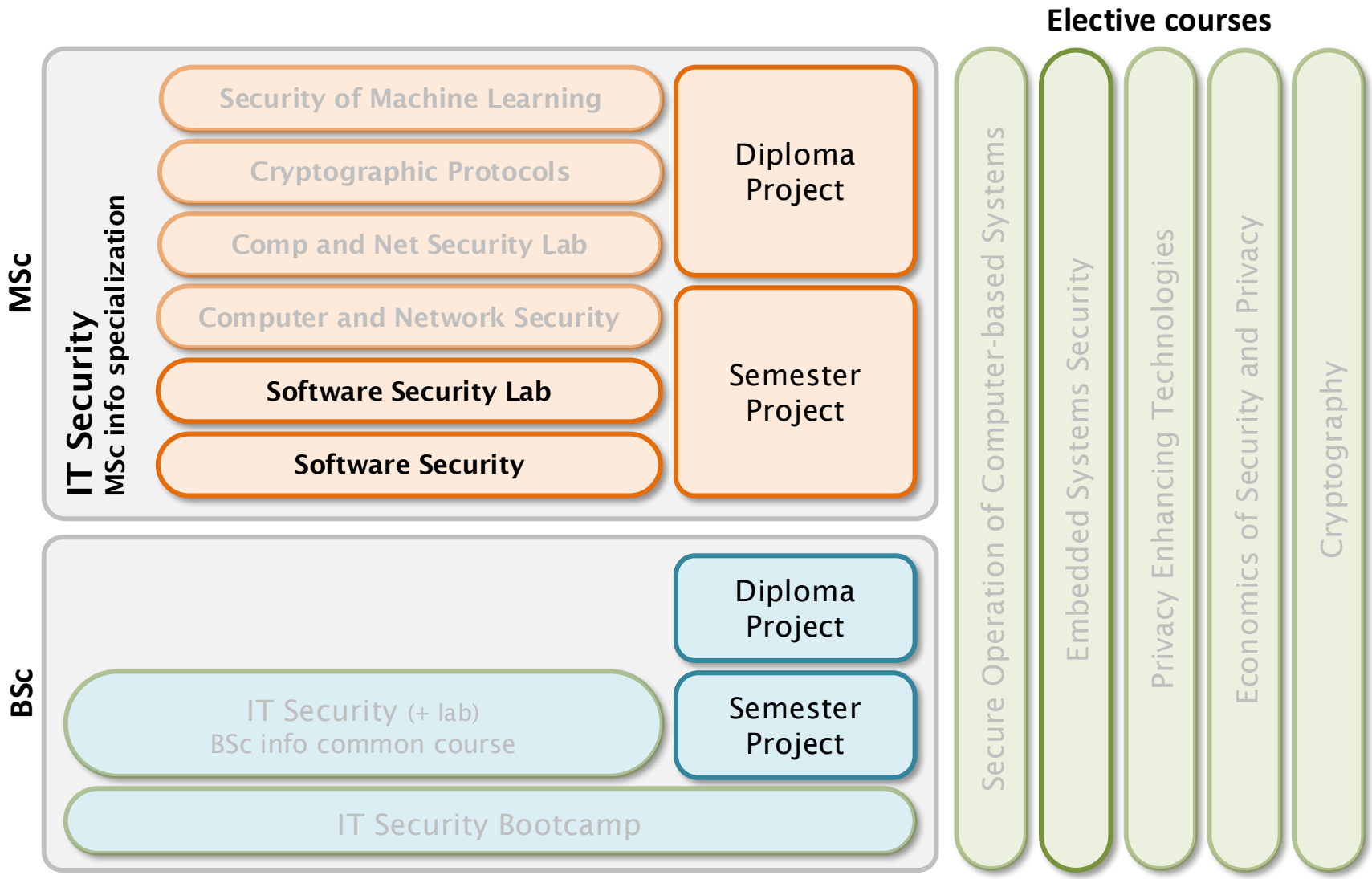
Összefoglaló

- Sok területet érintettünk ma:
  - Egyszerű puffertúlcsordulások
    - » Alapötlet és valósághű körülmények
    - » Shellkód összeállítása
  - Védelmi intézkedések
    - » Kanárik, DEP, ASLR

- A statisztikák azt mutatják, hogy a különböző memória korrupciós támadások problémája velünk marad még egy jó ideig
  - Beágyazott rendszereket sok helyen még mindig C/C++ nyelven programozzák
  - Védelmi intézkedések sokszor nem érhetőek el kellő választékban

- Legjobb megoldás a helyes kód!

IT biztonság oktatási program (BSc, MSc)



more info: http://www.crysys.hu/education/

Ellenőrző kérdések

- Mely programozási nyelveket érinti leginkább a puffertúlcsordulás problémája?
- Mi az a stack frame? Hol helyezkednek el a stack-en a függvényparaméterek és a lokális változók?
- Mi a stack overflow alapötlete?
- Hova tudja a támadó a kódját beszúrni egy stack overflow támadásnál?
- Mi mást lehet felülírni egy stack overflow támadásban a visszatérési cím mellett?
- A stack overflow mellett milyen más memória-korrupciós támadásokat ismer?

Ellenőrző kérdések

- Mi az a shellkód?
- Mi az a NOP sled? Miért alkalmazzák?
- Miért kell kerülni a 0x00 bájtokat a shellkódokban? Hogyan kerülhetők el?
- Milyen védelmi intézkedéseket ismer a stack overflow ellen?
- Hogy nehezíti meg egy stack overflow kihasználását a kanári alapú védelem?